factwise.io

Assessment: Technical Product Manager Campus

- This assessment has two sections and the document has 2 pages
- You are expected to submit within 2.5 hours of receiving this file
- Please make the submission as a PDF
- Your file should be named 'Your Name TPM' (e.g.: John Smith TPM)

Problem Statement: You are tasked with building an app for a virus quarantine & treatment facility From

the head doctor:

"This virus has a 15% mortality rate, and its main symptom is a prolonged fever. A patient is considered cured once they have reported 3 full days without a fever. Currently, our nurses jot down temperatures in a journal. The doctor reviews these journals when deciding what treatment to administer and informs the admin staff when they feel a patient could be discharged (solely based on the 3-day no-fever criterion).

We are understaffed and unable to keep up with the volume of patients coming in. We have rooms for up to 74 patients, which are currently at full occupancy.

Our process requires that each patient have their temperature measured by a nurse daily, and then be visited by a doctor at some point on the same day.

However, in reality, no one knows which patients have had their tempe

ratures measured, and which patients have been visited by a doctor already. So, sometimes a patient's temperature is measured multiple times on the same day, and sometimes their temperature is not measured at all. Doctor visits are also similarly erratic, and sometimes the doctor visits a patient before their temperature has been taken – as a consequence, the doctor has to waste precious minutes on doing this task themself or come back later. At times, patients end up staying an extra day or two in our centre because of these inefficiencies, or because doctors fail to promptly inform admin staff about discharges.

We want to treat as many patients as possible, so we need to be better organized, and discharge patients in a timely manner. We also want to track the quality of our treatment, so we can take measures in case the mortality rate in our centre rises above the expected 15%. Currently, we have no way of measuring our performance against the 85% success rate benchmark."

You and a small team of developers need to build an app to help this centre with its temperature tracking and administration needs. Your team needs to deliver a working product ASAP to solve the centre's basic requirements, and *nice-to-have features can be prioritized later*.

The content of this document is confidential and intended for the recipient specified in the message only. It is strictly forbidden to share any part of this document with any third party, without a written consent of the sender. If you received this message by mistake, please reply to this message and follow with its deletion, so that we can ensure such a mistake does not occur in the future.

Q1) In your app's first version, what are the different kinds of users, and which features and actions are relevant for which user types?

Q2) Please build a functional prototype of the app using any AI tool of your choice. Please provide a deployable working link either on vercel or any platform of your choice.
Your submission should include:

- **All basic screens** of the app (UI flow).
- **Working functionality for key actions**. For example:
  - A nurse should be able to **add and record a patient's temperature**.
  - Any other core features relevant to the app should be demonstrable.

Please ensure the prototype shows both the **screen designs** and the **actual workflow** for each feature.

Q3) For the entire app overview, what additional logical constraints would you provide to the developers? Include details not immediately apparent from the screen designs, such as when to display warnings or errors to the user, and when to allow or disallow certain actions for specific user types, etc.

Q4) In the next sprint planning meeting, after reviewing the last sprint, you and your team estimate that their velocity is 25 story points. For the next sprint, you already plan to finish 3 stories of 5 points each. In addition to these, which of the following options would you add to the sprint such that the effort is minimized and the value is maximized?

Assume that High priority features are 2X as valuable
as medium priority features, and medium priority features are 3X as valuable as low priority features.

Feature	Priority	Effort
A	High	1
B	High	3
C	High	5
D	Medium	2
E	Medium	3
F	Medium	1
G	Low	0.4
H	Low	0.5
I	Low	0.3
J	Low	0.1

The content of this document is confidential and intended for the recipient specified in the message only. It is strictly forbidden to share any part of this document with any third party, without a written consent of the sender. If you received this message by mistake, please reply to this message and follow with its deletion, so that we can ensure such a mistake does not occur in the future.



The content of this document is confidential and intended for the recipient specified in the message only. It is strictly forbidden to share any part of this document with any third party, without a written consent of the sender. If you received this message by mistake, please reply to this message and follow with its deletion, so that we can ensure such a mistake does not occur in the future.

FactWise — Technical Product Manager Campus Assessment

Candidate: Parth | **Prototype (deployable link):** [deployed link pending]

Q1 — User types, and which features/actions apply to which users

The app's first version has three user types, matching the three distinct jobs described in the brief (measuring, treating, and administering):

1. Nurse

The nurse's job is purely data capture — they should not be making discharge or treatment calls.

- **Record a patient's temperature** (the core action the brief explicitly calls out). Each entry is timestamped and tagged with who recorded it.
- **See a daily worklist** of which active patients still need today's temperature taken, and which are already done — this directly fixes the facility's stated problem ("no one knows which patients have had their temperature measured").
- **View a patient's temperature history**, with fevers visually flagged.
- Cannot visit, discharge, admit, or mark a patient deceased.

2. Doctor

The doctor owns the clinical decision — treatment and the discharge call (per the brief: "the doctor reviews these journals... and informs the admin staff").

- **See a daily rounds list**: who hasn't been visited yet today vs. who has, so visits aren't duplicated or skipped.
- **View each patient's full temperature history and an auto-computed fever-free streak** (replaces manually reading the paper journal).
- **Log a visit** for a patient (with optional treatment notes), which marks them "visited today."
- **See a system-generated discharge recommendation** once 3 consecutive fever-free days are on record, and flag the patient for admin — the system computes eligibility, but a human (the doctor) still has to act on it, since the brief is explicit that this is the doctor's call.
- **Mark a patient as deceased** (clinical outcome, feeds the facility's mortality tracking).
- Cannot admit or formally discharge a patient — bed/room state belongs to admin.

3. Admin

Admin owns facility operations — beds, admissions, and the performance numbers the head doctor asked for.

- **Facility dashboard**: current occupancy vs. the 74-bed capacity, discharged/deceased counts, and success rate vs. the 85% benchmark.
- **Admit a new patient** (blocked once the facility is at capacity; room/bed numbers are validated so two active patients can't share a room).

- **See the doctor-approved discharge queue and confirm discharge**, which frees the bed. This is a deliberate second step — it mirrors the two-person handoff in the brief and ensures a bed is never freed without medical sign-off.
- Cannot record temperatures, log visits, or mark a patient deceased — those are clinical actions.

(Nice-to-have for later versions: authenticated logins per staff member, shift assignment, editing a mis-entered reading, notifications, multi-facility support. The deployed prototype also demonstrates a richer vitals/escalation layer beyond this MVP — see the note at the end of this document.)

Q3 — Additional logical constraints for developers

Things that aren't obvious from the screens themselves, but which the logic needs to get right:

1. **Fever threshold.** A reading counts as a fever at ≥100.4°F (38°C). This single constant drives every downstream decision (streaks, badges, warnings) — it should live in one place, not be hard-coded per screen.
2. **What “3 full days without a fever” means.** It's 3 consecutive calendar days, each of which (a) has at least one recorded reading, and (b) has no reading at/above the fever threshold. A day with zero measurements should break/interrupt the streak, not be silently skipped — otherwise the app could recommend discharging a patient who simply wasn't checked. The UI must surface this as a warning (“gap in temperature log”) rather than hide it.
3. **“Today” is never auto-counted as fever-free.** Until a nurse actually logs today's reading, today doesn't count toward (or break) the streak — it's just not yet knowable.
4. **Discharge is a recommendation, not an automatic action.** Once the 3-day criterion is met, the system flags it — but it only moves into admin's queue after the doctor explicitly acts (logs today's visit / reviews it). The algorithm should never discharge a patient on its own, because the brief states this is solely the doctor's judgment call.
5. **Discharge requires two sign-offs.** Doctor approval → admin confirmation. This models the real handoff in the brief and is also what actually frees the bed — it should be impossible to free a bed without both steps.
6. **Bed capacity is a hard ceiling (74).** New admissions must be blocked outright once full, not just warned. A room/bed number in use by another active patient cannot be reused until that patient is discharged or deceased.
7. **Discharged/deceased patients become read-only.** No further temperature entries, visits, or discharge actions should be possible against a closed record.
8. **Sanity-bound temperature entries.** The system should accept multiple readings a day (re-checks happen) but reject/warn on physiologically implausible values (e.g., outside ~90–110°F) rather than silently accepting a typo.
9. **Warnings vs. hard blocks.** “No temperature recorded today” or “not yet visited today” should be a visible warning that doesn't block other actions (so work can continue); capacity-full and duplicate-room/bed are hard blocks that prevent submission entirely.
10. **Role-based write access.** Nurses can only write temperature records, doctors only write visit/outcome records, admins only write admission/discharge-confirmation records. This isn't just UI hiding — it should be enforced wherever the data is written, since mixing these up is exactly the kind of accountability gap the facility is already struggling with.
11. **Mortality/success-rate benchmark (85%) is computed only over resolved cases** (discharged + deceased). Active, still-being-treated patients shouldn't dilute a live rate that's meant to measure treatment outcomes, and admin's view should flag it red the moment it dips below 85%.
12. **Audit trail.** Every temperature reading and visit should retain who recorded it and when — this is the direct fix for the facility's stated problem that “no one knows” who did what.

Q4 — Sprint planning

Setup. Team velocity = 25 points. Already committed: 3 stories × 5 points = 15 points. **Remaining capacity = 25 − 15 = 10 points.**

Value is relative: High = 2× Medium, Medium = 3× Low. Assigning Low = 1 unit, Medium = 3 units, High = 6 units satisfies both ratios.

Feature	Priority	Effort	Value	Value / Effort
A	High	1	6	6.0
B	High	3	6	2.0
C	High	5	6	1.2
D	Medium	2	3	1.5
E	Medium	3	3	1.0
F	Medium	1	3	3.0
G	Low	0.4	1	2.5
H	Low	0.5	1	2.0
I	Low	0.3	1	3.3
J	Low	0.1	1	10.0

This is a knapsack problem: maximize total value subject to total effort ≤ 10. Ranking by value/effort and greedily filling (then checking no swap improves on it) gives one clear winner:

Add: A, B, D, F, G, H, I, J. Skip: C and E.

- Total effort = 1 + 3 + 2 + 1 + 0.4 + 0.5 + 0.3 + 0.1 = **8.3 points** (fits inside the 10-point remaining capacity)
- Total value = 6 + 6 + 3 + 3 + 1 + 1 + 1 + 1 = **22 units** — the maximum achievable within 10 points of effort (verified: no other combination, including swapping in C or E, reaches higher value without exceeding capacity; using D instead of E to reach the same value also uses less effort, which is why E is dropped and D is kept).
- 1.7 points of capacity are left over — there's no item small enough left (C needs 5, E needs 3) to use it without displacing something already in the optimal set, so it's intentionally left unused rather than overcommitting the team.

Why not just take everything by priority? C (High, but effort 5 for only 6 value) and E (Medium, 3 effort for only 3 value) both have the worst value-to-effort ratios in the table — including them would either blow the 10-point budget or force dropping multiple cheaper, higher-ratio items, reducing total value delivered this sprint.

Note on the prototype's scope: the deployed app goes beyond what this brief strictly asks for — it adds SpO2, heart rate, blood pressure, respiratory rate, an AVPU neuro scale, and a nurse→doctor escalation/paging workflow, on top of the core temperature-and-discharge tracking described above. That was a deliberate choice to make the demo more complete, not a claim that it's all MVP — my answer to Q1 above is what I'd actually ship first; the vitals/escalation layer is the clearest “nice-to-have, prioritize later” bucket per this brief's own instruction.